

Architecture U-Net

Segmentation avec et sans attention

*Encodeur-décodeur · Skip connections · Attention Gate · Variantes Transformer · Applications
HTR*

U-Net — omniprésent dans le pipeline HTR

Cette section fait la synthèse de l'encodeur-décodeur (section 2.0) et de l'attention (section 2.2)

U-Net dans le pipeline manuscrits médiévaux

Binarisation des pixels d'encre

→ Un U-Net appris gère les cas difficiles (show-through, parchemin dégradé) que Sauvola ne peut pas

Segmentation de lignes (BLLA de Kraken)

→ Le module BLLA est lui-même une architecture U-Net convolutif spécialisé

Segmentation de layout (dhSegment)

→ Outil standard pour les documents historiques : U-Net avec encodeur ResNet

Séparation texte / illustration

→ Segmentation sémantique binaire ou multi-classes (ontologie SegmOnto)

Comprendre U-Net en profondeur, c'est comprendre le fondement technique de plusieurs étapes clés du pipeline.

Pourquoi une section dédiée ?

Section 2.0 : U-Net présenté comme référence pour la segmentation

Section 2.2 : mécanisme d'attention étudié en détail

Cette section : synthèse — pourquoi U-Net fonctionne si bien, et comment l'attention l'améliore qualitativement

Citation clé

« La segmentation est une tâche de traduction : traduire chaque pixel en une décision.

U-Net a compris que la bonne traduction ne sacrifie pas l'original — elle le conserve intégralement le long du chemin, pour y revenir quand c'est nécessaire. »

1. La tension fondamentale de la segmentation

Compréhension sémantique vs localisation pixel précise

La tension spatiale et la solution de U-Net

Classification → compresser · Segmentation → préserver · La contradiction est structurelle

La tension fondamentale

Besoin 1 — Compréhension sémantique haute

→ nécessite de la PROFONDEUR et de la COMPRESSION

→ tue la résolution spatiale

Besoin 2 — Localisation pixel précise

→ nécessite de la RÉOLUTION et de ne PAS compresser

→ empêche la profondeur sémantique

FCN (approche brute) : comprime jusqu'au bottleneck, upsample directement

→ Sémantiquement correct mais spatialement flou aux bords

La solution de U-Net — connexions latérales

Maintenir des connexions latérales (skip connections) entre encodeur et décodeur à chaque niveau de résolution

Information haute résolution (encodeur)

→ transmise directement au décodeur (même niveau)

Information sémantique profonde (bottleneck)

→ remonte par le chemin du décodeur

Le décodeur a accès aux deux simultanément

Son seul travail : sélectionner ce qui est pertinent

Implication pour les manuscrits médiévaux

Sans skip : un réseau qui a réduit $512 \times 512 \rightarrow 16 \times 16$ a perdu la résolution pour distinguer les pixels d'un jambage de ceux de la ligne adjacente

Avec skip : chaque niveau (texture parchemin, contours de traits, formes locales, mots) est conservé et réinjecté exactement là où il est utile

→ Précision des contours de segmentation incomparable à ce que FCN peut produire depuis le bottleneck seul

2. Architecture U-Net standard

Encodeur · Bottleneck · Décodeur · Skip connections · Loss

U-Net — Vue d'ensemble de l'architecture

La forme en U : résolution en abscisse, profondeur en ordonnée (Ronneberger et al., MICCAI 2015)

Encodeur (descente)

Bloc 1 $\rightarrow H/2 \times W/2 \times 64$
Conv $\times 2$ + ReLU + BN +
MaxPool

Bloc 2 $\rightarrow H/4 \times W/4 \times 128$
Conv $\times 2$ + ReLU + BN +
MaxPool

Bloc 3 $\rightarrow H/8 \times W/8 \times 256$
Conv $\times 2$ + ReLU + BN +
MaxPool

Bloc 4 $\rightarrow H/16 \times W/16 \times 512$
Conv $\times 2$ + ReLU + BN +
MaxPool

Bottleneck + Décodeur

Bottleneck $\rightarrow H/16 \times 1024$
Conv $\times 2$ + ReLU + BN (pas de
pool)

Bloc 6 $\uparrow \rightarrow H/8 \times 512$
TranspConv $\uparrow$ + Concat(skip4)
+ Conv $\times 2$

Bloc 7 $\uparrow \rightarrow H/4 \times 256$
TranspConv $\uparrow$ + Concat(skip3)
+ Conv $\times 2$

Bloc 8 $\uparrow \rightarrow H/2 \times 128$
Bloc 9 $\uparrow \rightarrow H \times 64$
Tête : Conv $1 \times 1 \rightarrow H \times W \times$
n_classes

Les skip connections — le coeur de U-Net

4 skip connections : encodeur $\rightarrow$ décodeur au même niveau de
résolution

Implémentées par concaténation sur la dimension des canaux (pas
d'addition)

Concaténation $\rightarrow$ le décodeur voit les features encodeur ET décodeur
 $\rightarrow$ peut décider comment les combiner

Brique de base — Double bloc convolutif

Conv $3 \times 3 \rightarrow$ BatchNorm $\rightarrow$ ReLU

Conv $3 \times 3 \rightarrow$ BatchNorm $\rightarrow$ ReLU

Deux convolutions $3 \times 3 \approx$ une convolution 5×5 en champ réceptif
mais 2 $\times$ moins de paramètres + 2 non-linéarités

~ 31 M paramètres au total pour features=[64,128,256,512]

Traite des images de taille arbitraire (réseau entièrement convolutif)

Upsampling et loss de segmentation

Deux stratégies de montée en résolution · BCE + Dice pour équilibrer précision locale et recouvrement global

Upsampling — deux approches

1. Upsampling bilinéaire + Conv 1×1 (préférable pour les manuscrits)
 - Simple, pas de paramètres supplémentaires
 - Artefacts limités, recommandé en pratique
2. Convolution transposée (déconvolution)
 - Upsampling appris, plus expressif
 - Peut produire des artefacts en damier si mal initialisée
 - stride=2 : résolution × 2

BCE + Dice Loss — combinaison recommandée

Binary Cross-Entropy (BCE / Cross-Entropy) :

- Pénalise chaque pixel indépendamment
- Sensible aux déséquilibres de classes

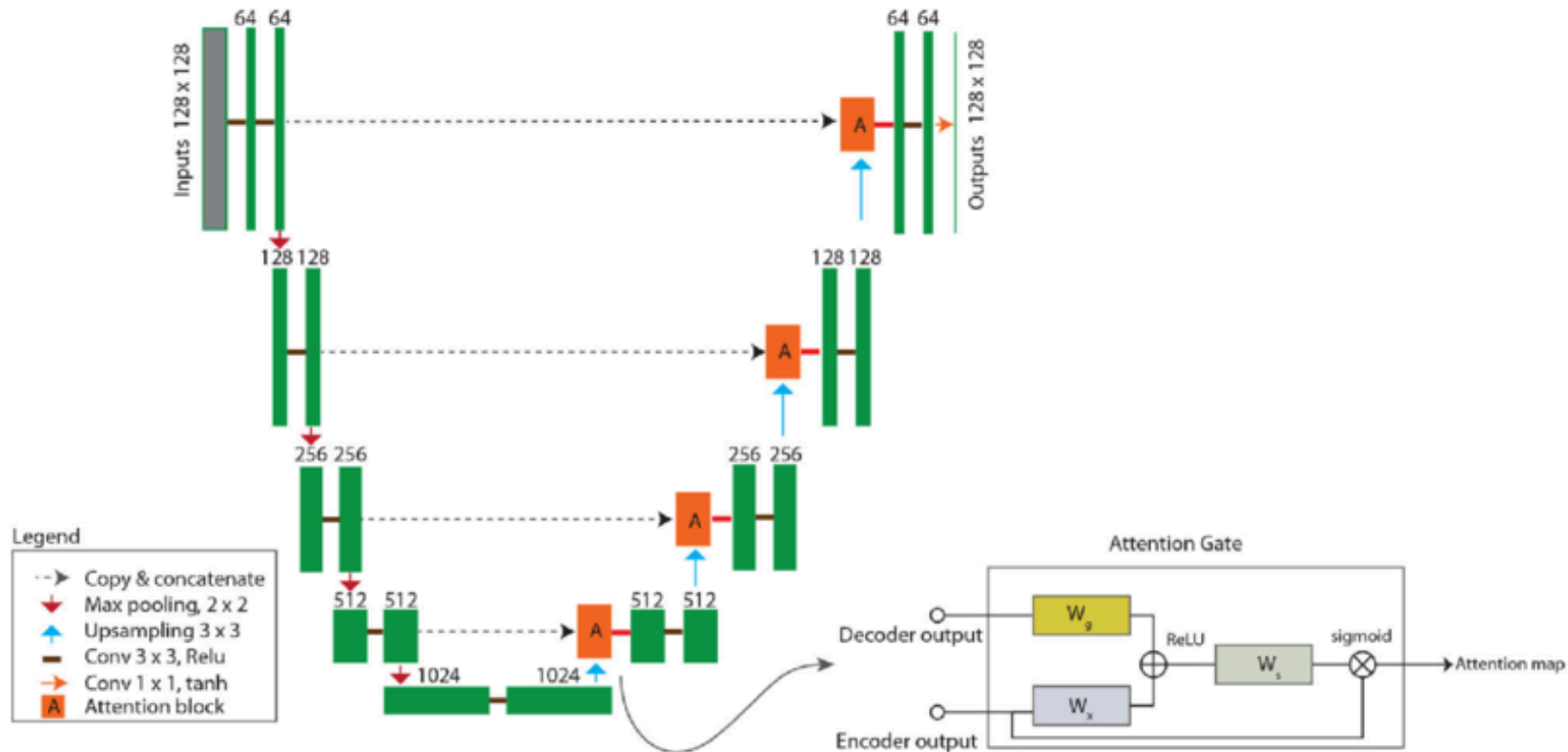
Dice Loss : $L_{Dice} = 1 - 2|TP| / (|pred| + |ref|)$

- Mesure le recouvrement global des masques
- Robuste aux déséquilibres (pixels d'encre = 10-20% du scan)

Combinaison : $L = \alpha \cdot BCE + \beta \cdot Dice$ ($\alpha=\beta=0.5$ par défaut)

Pourquoi combiner les deux losses pour les manuscrits ?

BCE seule : optimise pixel par pixel → excellent pour les zones denses (corps du texte), mais ignore le recouvrement global des masques de lignes
Dice seul : optimise le recouvrement → robuste aux déséquilibres, mais peut sous-pénaliser les erreurs isolées (pixels d'ascendants fins)
BCE + Dice : équilibre précision locale (bords de jambages, ascendants) et recouvrement global (masques de lignes entières)



U-Net avec attention

3. U-Net et les manuscrits médiévaux

Hiérarchie des features · Données limitées · Équivariance spatiale

Hiérarchie des features dans un scan de manuscrit

Chaque niveau d'encodage capture une échelle d'information différente — toutes nécessaires pour la segmentation

Niveau 1 — 512×512

Textures locales
Rugosité du parchemin
Grain de l'encre
Micro-variations de couleur

Niveau 2 — 256×256

Contours des traits
Bords des jambages
Zones de transition
encre/fond

Niveau 3 — 128×128

Formes locales
Ascendants et boucles
Ligatures inter-lettres

Niveau 4 — 64×64

Patterns de mots
Espacement inter-mot
Zones denses de texte

Bottleneck — 32×32

Structure globale de la page
Présence d'illustration
Densité de colonnes
Position des marges

Pourquoi U-Net est unique — tous les niveaux sont nécessaires simultanément

Le bottleneck décide si une région est du texte ou une illustration ·
Les niveaux intermédiaires localisent les lignes · Les niveaux haute
résolution précisent les contours

Sans skip connections : le décodeur doit reconstruire tous ces niveaux depuis le
bottleneck seul — ce qu'il fait approximativement et imprécisément
Robustesse en données limitées : l'info de localisation n'est pas apprise depuis
zéro — elle est transmise directement → modèle compétitif avec quelques
dizaines d'images

4. Attention U-Net — gating spatial

Filtrer les skip connections selon le contexte sémantique du décodeur

Le problème des skip connections non filtrées

U-Net standard transmet TOUTES les features de l'encodeur — y compris le bruit, les dégradations, les taches

Le problème — propagation du bruit

U-Net : chaque skip connection transmet toutes les features

- Les features de la tache sont transmises fidèlement au décodeur
- Le décodeur doit apprendre à les ignorer pendant la segmentation de l'encre
- Les skip connections non filtrées propagent du bruit vers la prédiction finale
- Erreurs de segmentation dans les zones dégradées

La solution — Attention U-Net (Oktay et al., 2018)

Mécanisme de gating sur les skip connections

- Avant la concaténation, les features de l'encodeur sont filtrées
- Un score d'attention $\alpha_{ij} \in [0,1]$ pondère chaque pixel

Score calculé en combinant :

g : contexte sémantique du décodeur (« que suis-je en train de segmenter ? »)

x : contenu de la skip connection (ce qu'on cherche à filtrer)

- Un pixel de tache reçoit $\alpha \approx 0$: atténué avant la concaténation
- Un pixel d'encre pertinent reçoit $\alpha \approx 1$: transmis intégralement

Analogie avec la self-attention (section 2.2)

Attention Gate $\equiv$ cross-attention entre le décodeur (Query) et la skip connection (Key/Value)

Le décodeur « interroge » l'encodeur : parmi tous les pixels de ce niveau, lesquels sont utiles pour la tâche courante ?

Différence avec la self-attention globale : l'AG est local et spatial (pixel par pixel), pas un produit scalaire sur toute la séquence

Le mécanisme d'Attention Gate

$$\alpha_{ij} = \sigma(W_{\psi} \cdot \text{ReLU}(W_g \cdot g_{ij} + W_x \cdot x_{ij} + b)) \cdot \hat{x}_{ij} = \alpha_{ij} \cdot x_{ij}$$

Les entrées du gate

g — Vecteur de gating (décodeur)

→ Basse résolution $H \times W$

→ Haute sémantique C_g canaux

→ Encode : « que suis-je en train de segmenter ? »

x — Skip connection (encodeur)

→ Haute résolution $H \times W$

→ Faible sémantique C_x canaux

→ Encode : le contenu à filtrer

Les deux sont à des résolutions différentes

→ g est upsampled à la résolution de x avant la fusion

Le calcul en 3 étapes

1. Projections linéaires (Conv 1×1)

$W_g \cdot g \in \mathbb{R}^{C_{int}}$ (compresser g)

$W_x \cdot x \in \mathbb{R}^{C_{int}}$ (compresser x)

$C_{int} = C_x / 2$ (dimension intermédiaire)

2. Addition + ReLU

$att = \text{ReLU}(W_g \cdot g + W_x \cdot x + b)$

→ Combine contexte sémantique et contenu local

3. Score scalaire par pixel + Sigmoid

$\alpha = \text{Sigmoid}(W_{\psi} \cdot att) \in [0, 1]^{(H \times W)}$

$W_{\psi} \in \mathbb{R}^{1 \times C_{int}} \rightarrow$ un scalaire par pixel

Sortie filtrée :

$\hat{x}_{ij} = \alpha_{ij} \cdot x_{ij}$ (pondération pixel par pixel)

Interprétation sur les manuscrits

Contexte : segmenter les baselines (lignes de base)

g encode : « je cherche des structures horizontales linéaires »

Traits d'écriture horizontaux $\rightarrow \alpha \approx 1$

→ Transmis intégralement au décodeur

Taches d'humidité $\rightarrow \alpha \approx 0$

→ Atténués avant la concaténation

Ascendants verticaux $\rightarrow \alpha$ intermédiaire

→ Partiellement transmis

Carte α visualisable : outil d'interprétabilité

Cartes d'attention et comparaison avec U-Net standard

La carte α_{ij} révèle exactement sur quoi le modèle se concentre à chaque niveau du décodeur

Cartes d'attention — interprétabilité directe

$\alpha_{ij} \in [0,1]$ est directement visualisable comme une carte de chaleur

Niveau décodeur 1 (proche sortie, haute résolution) :

→ *Attention concentrée sur les traits d'encre fins et les contours*

Niveau décodeur 2 (résolution intermédiaire) :

→ *Attention sur les structures de lettres et ligatures*

Niveau décodeur 3-4 (proche bottleneck, basse résolution) :

→ *Attention sur les zones de texte denses vs illustrations*

Superposition $\alpha \times \text{image}$: visualisation des zones pertinentes pour la segmentation courante

Différences clés U-Net vs Attention U-Net

U-Net standard :

- Toutes les features de l'encodeur sont concaténées telles quelles
- Le décodeur apprend à ignorer le bruit — mais y parvient imparfaitement
- Erreurs sur les images très dégradées

Attention U-Net :

- Seules les features $\alpha \cdot x$ sont concaténées (filtrées)
- Le bruit est atténué avant la concaténation
- Meilleure précision sur les zones dégradées
- Surcoût : ~4M paramètres supplémentaires (~35M vs ~31M)

Quand préférer Attention U-Net sur les manuscrits ?

Images très dégradées : show-through visible, taches importantes, encre corrodée ou effacée → AG atténue les features de dégradation
Segmentation multi-classes avec classes très déséquilibrées (lettrines rares vs texte abondant) → AG focalise sur les zones discriminantes
Données suffisantes (> 200 images annotées) pour apprendre les weights W_g , W_x , W_ψ de façon fiable

5. Variantes modernes — U-Net + Transformers

TransUNet · Swin U-Net · SegFormer — combiner localité CNN et globalité Transformer

TransUNet, Swin U-Net et SegFormer

Trois stratégies pour intégrer les Transformers dans une architecture encodeur-décodeur

TransUNet (Chen, 2021)

CNN encodeur (ResNet) → features locales robustes

Bottleneck → encodeur Transformer
→ Self-attention globale exactement là où c'est utile

Skip connections CNN → décodeur CNN
~105M paramètres · mIoU 44% (ADE20k)

Swin U-Net (Cao, 2022)

Tous les blocs → Swin Transformer (encodeur, bottleneck, décodeur)

Skip connections Transformer → Transformer

Champ réceptif global dès la 1ère couche
Complexité linéaire (fenêtres décalées)
~41M paramètres · mIoU 44% (ADE20k)

SegFormer (Xie, 2021)

Encodeur Mix Transformer (MiT) hiérarchique

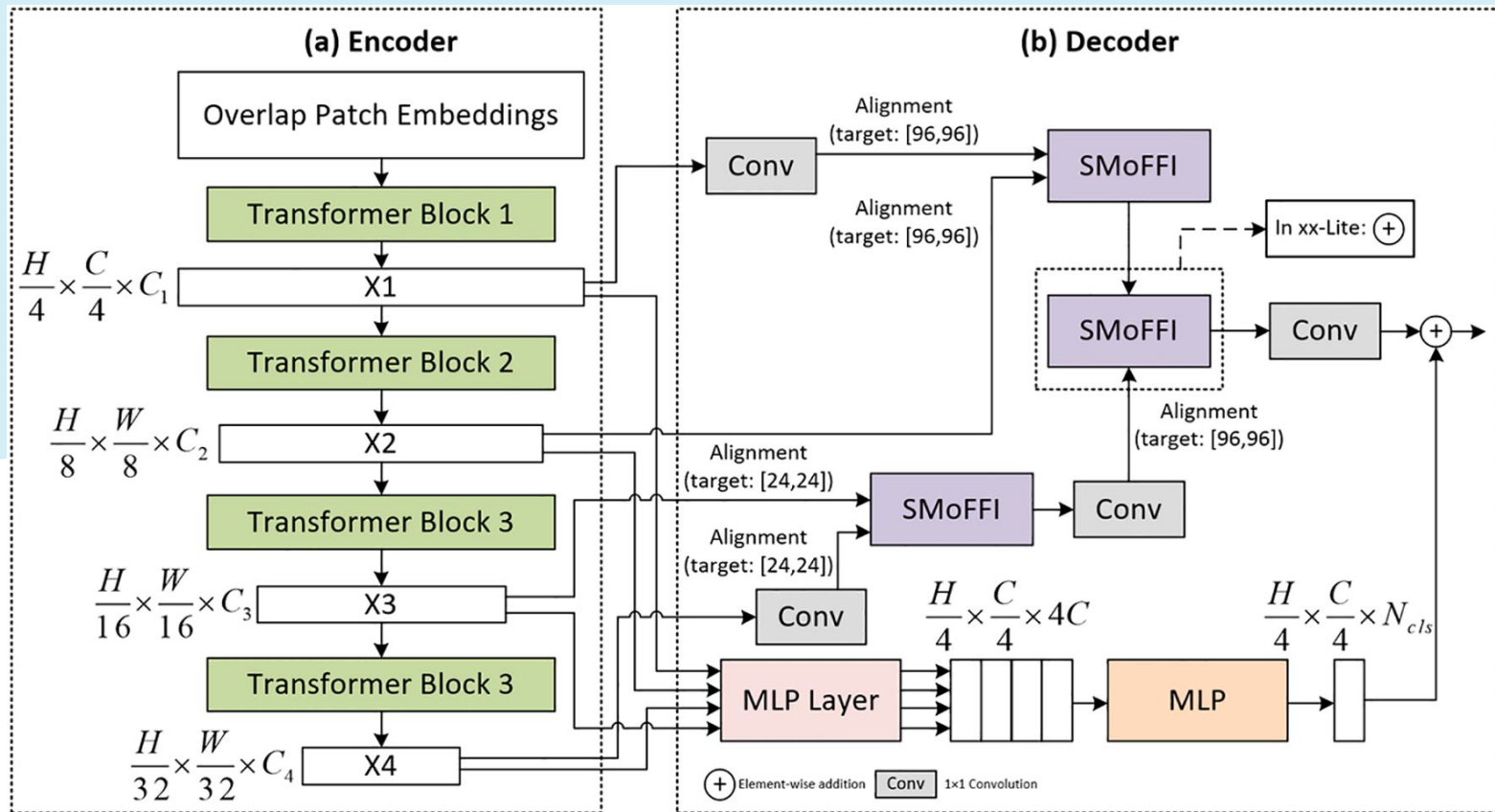
Décodeur MLP léger — pas de convolutions

Architecture la plus légère et performante

~82M (B5) · mIoU 51% (ADE20k)

Recommandé si la mémoire GPU est limitée

Architecture	Encodeur	Bottleneck	Décodeur	Paramètres	mIoU ADE20k
U-Net	CNN	CNN	CNN	~31M	~37%
Attention U-Net	CNN+AG	CNN	CNN	~35M	~40%
TransUNet	CNN	Transformer	CNN	~105M	~44%
Swin U-Net	Swin	Swin	Swin	~41M	~44%
SegFormer-B5	MiT	—	MLP	~82M	~51%



Variante de SegFormer

H	Hauteur de l'image
W	Largeur de l'image
C	Nombre de canaux
SMoFFI	S patial M ixing of F eatures with F used Information

6 & 7. Applications et entraînement pratique

Binarisation · Layout SegmOnto · BLLA Kraken · Transfert & stratégie progressive

Applications U-Net aux manuscrits médiévaux

Trois tâches concrètes dans le pipeline HTR — chacune avec ses spécificités de configuration

Binarisation adaptative

Tâche : fond (0) vs encre (1) —
segmentation binaire

Configuration recommandée :

in_channels=3 (RGB, pas niveaux de gris)

→ L'encre ferro-gallique brunie a un
spectre distinct du fond

n_classes=2

features=[32,64,128,256] (modèle léger)

dropout=0.2 (données limitées)

Données : paires (scan RGB, masque
DIBCO)

vs Sauvola : amélioration 1-3 pts CER sur
parchemins dégradés

Segmentation de layout — SegmOnto

7 classes : MainZone, MarginTextZone,
DropCapitalZone, GraphicZone,
TitlePageZone, NumberingZone,
Background

→ Attention U-Net recommandé
L'AG distingue zones de texte dense (att
forte) des zones de fond (att faible) sans
être perturbé par les ornements

in_channels=3 · n_classes=7

features=[64,128,256,512]

Plus précis que SAM sur les pages à mise
en page standard annotée

BLLA de Kraken — détection de baselines

Le module BLLA est lui-même un réseau de type
U-Net convolutif entraîné sur des documents
patrimoniaux

Tâche de sortie : carte de pixels de baselines +
carte de régions

Pourquoi U-Net est adapté :

→ Baselines : structures fines (quelques pixels) →

skip connections nécessaires pour la précision

→ Détection globale (nb colonnes, illustrations)

→ bottleneck sémantique nécessaire

Les deux besoins simultanément → U-Net
naturellement adapté

Entraînement pratique sur des données de manuscrits

< 1000 images annotées → overfitting inévitable sans transfert et stratégie progressive

Transfert depuis ImageNet — encodeur pré-entraîné

Problème : entraîner U-Net de zéro sur < 1000 images → overfitting

Solution : encoder ResNet pré-entraîné comme backbone
(segmentation-models-pytorch facilite cette combinaison)

```
smp.Unet(encoder_name='resnet34', encoder_weights='imagenet')
```

Stratégie d'entraînement progressive en 2 étapes :

Étape 1 (5 epochs) : geler l'encodeur
→ Entraîner uniquement le décodeur
→ LR élevé pour le décodeur (1e-4)

Étape 2 (20 epochs) : dégeler tout
→ LR différenciés : encodeur 1e-5, décodeur 1e-4
→ L'encodeur pré-entraîné est affiné doucement

Initialisation de He (Kaiming) — couches aléatoires

Parties initialisées aléatoirement (décodeur, tête) : initialisation de He
Conv2d : `kaiming_normal_(mode='fan_out', nonlinearity='relu')`
BatchNorm : `weight=1, bias=0`
Recommandée pour les réseaux avec ReLU — évite la saturation au début

Augmentations recommandées pour les manuscrits

Déformations élastiques : simulent la courbure du parchemin, les déformations de support — très efficaces (Ronneberger, 2015)
Flips horizontaux : prudence — écriture manuscrite non symétrique
Variations de luminosité/contraste : simulent les dégradations d'encre
Rotations légères ($\pm 5^\circ$) : lignes légèrement inclinées
Bruit gaussien : simule le grain du parchemin

Récapitulatif — U-Net et Attention U-Net

U-Net standard

Encodeur-décodeur symétrique
Skip connections par concaténation

Brique : Conv $3 \times 3 \times 2$ + BN + ReLU

Upsampling bilinéaire + Conv 1×1

Loss : BCE + Dice
~31M paramètres

Forces :
→ Précision de segmentation
→ Données limitées
→ Taille d'image arbitraire

Attention U-Net

Ajoute un Attention Gate sur chaque skip connection

$\alpha_{ij} = \sigma(W_\psi \cdot \text{ReLU}(W_g \cdot g + W_x \cdot x))$

Filtre les features de l'encodeur selon le contexte sémantique du décodeur
~35M paramètres

Force supplémentaire :
→ Atténuation du bruit et des dégradations
→ Interprétabilité (carte α)
→ Préféré pour les données très dégradées

Variantes Transformer

TransUNet : CNN + Transformer bottleneck
→ 105M · mIoU 44%

Swin U-Net : Transformer pur en U
→ 41M · mIoU 44%
→ Compatible FPN, Mask R-CNN

SegFormer : MiT + MLP décodeur
→ 82M · mIoU 51% — le plus performant

Tendance : encodeur Transformer + skip connections conservées

Applications HTR

Binarisation
→ U-Net RGB léger
→ vs Sauvola : +1-3 pts CER

Layout SegmOnto
→ Attention U-Net
→ 7 classes ontologie

BLLA Kraken
→ U-Net convolutif spécialisé
→ Baselines + polygones

Transfert : ResNet-34 ImageNet
→ Stratégie en 2 étapes